



دانشگاه صنعتی شریف
دانشکده مهندسی کامپیوتر

گزارشکار پروژه درس

عنوان:

گزارش پروژه: پردازنده MIPS دو حالت با قابلیت راه اندازی امن (Secure Boot)

اعضای گروه

نگار یار احمدی

فاطمه پرویزی

زهرا امیر بیگی

نیکی رشیدیان

نام درس

معماری کامپیوتر

نیم سال دوم ۱۴۰۵-۱۴۰۴

نام استاد درس

جناب آقای دکتر اسدی

فهرست مطالب

۱	چکیده	۱
۱	مقدمه و فروض مسئله	۲
۲	نحوه پیاده‌سازی و معماری کلی سیستم	۳
۲	نقشه حدودی کار	۴
۳	ماژول مسیر داده (DataPath)	۵
۳	۱-۵ توضیح کلی کد	۵
۳	۲-۵ کاربرد کد در سیستم	۵
۳	۳-۵ کد اصلی	۵
۸	۴-۵ بررسی کد ارائه‌شده	۸
۱۰	۶ پیاده‌سازی mode_bit	۱۰
۱۰	۱-۶ تعریف	۱۰
۱۰	۲-۶ پیاده‌سازی در DataPath	۱۰
۱۱	۳-۶ تولید سیگنال drop_priv_en در واحد کنترل	۱۱
۱۱	۴-۶ اتصال mode_bit در ماژول سطح بالا	۱۱
۱۲	۵-۶ کنترل دسترسی در روتر حافظه	۱۲
۱۳	۶-۶ نظارت توسط نگهبان سخت‌افزاری	۱۳
۱۳	۷-۶ خلاصه	۱۳
۱۳	۷ ماژول واحد کنترل (Control Unit)	۱۳
۱۳	۱-۷ توضیح کلی کد	۱۳
۱۴	۲-۷ کاربرد کد در سیستم	۱۴

۳-۷ کد اصلی ۱۴

۴-۷ بررسی کد ارائه شده ۱۹

۸ مازول روتر حافظه (Memory Router) ۲۱

۱-۸ توضیح کلی کد ۲۱

۲-۸ کاربرد کد در سیستم ۲۱

۳-۸ کد اصلی ۲۱

۴-۸ بررسی کد ارائه شده ۲۷

۹ مازول شتاب دهنده هش (Hash Accelerator) ۲۹

۱-۹ توضیح کلی کد ۲۹

۲-۹ کاربرد کد در سیستم ۲۹

۳-۹ کد اصلی ۲۹

۴-۹ بررسی کد ارائه شده ۳۱

۱۰ مازول نگهبان سخت افزاری (Hardware Watchdog) ۳۳

۱-۱۰ توضیح کلی کد ۳۳

۲-۱۰ کاربرد کد در سیستم ۳۳

۳-۱۰ کد اصلی ۳۳

۴-۱۰ بررسی کد ارائه شده ۳۵

۱۱ مازول سطح بالا (MIPS_Processor_Top) ۳۶

۱-۱۱ توضیح کلی کد ۳۶

۲-۱۱ کاربرد کد در سیستم ۳۶

۳-۱۱ کد اصلی ۳۶

۴-۱۱ بررسی کد ارائه شده ۴۱

۴۳	۱۲ یکپارچه‌سازی و تست‌بنچ نهایی (Top-Level Testbench)
۵۰	۱-۱۲ بررسی سناریوهای بالا
۵۱	۱۳ تست‌بنچ Hardware WatchDog Test Bench
۵۶	۱-۱۳ بررسی سناریوهای بالا
۵۷	۱۴ خروجی‌ها و نتایج شبیه‌سازی
۵۷	۱-۱۴ خروجی transcript
۵۸	۲-۱۴ خروجی موج
۵۹	۱۵ چالش‌ها و مشکلات پروژه
۵۹	۱-۱۵ چالش‌های اشکال‌زدایی تست‌بنچ راه‌اندازی امن
۶۰	۲-۱۵ چالش‌های افزودن واحد Hardware Execution Watchdog
۶۱	۳-۱۵ چالش‌های مرتبط با مسیر داده (DataPath)

فهرست تصاویر

۵۷		بخش اول خروجی transcript تست‌بنچ ماژول سطح بالا	۱
۵۷		بخش اول خروجی transcript تست‌بنچ ماژول سطح بالا	۲
۵۷		خروجی transcript بخش WatchDog	۳
۵۸		خروجی موج اول	۴
۵۸		خروجی موج دوم	۵

۱ چکیده

امنیت در سطح سخت‌افزار، پایه و اساس سیستم‌های نهفته‌ی قابل اعتماد است. در این پروژه، یک پردازنده MIPS پایه با افزودن قابلیت دوحالته بودن (Secure/User Mode) توسعه یافته است تا از اجرای کدهای مخرب جلوگیری کند. مکانیزم راه‌اندازی امن (Secure Boot) پیاده‌سازی شده در این سیستم، با استفاده از یک شتاب‌دهنده هش سخت‌افزاری، یکپارچگی برنامه‌ی کاربر موجود در حافظه را پیش از اجرا بررسی می‌کند. همچنین با طراحی یک روتر حافظه (Memory Router)، سطوح دسترسی به نواحی مختلف حافظه کنترل می‌شود تا کدهای حالت کاربر نتوانند به نواحی حساس سیستم نفوذ کنند. سیستم در نهایت پس از تایید هش، با استفاده از یک دستور اختصاصی سخت‌افزاری سطح دسترسی خود را کاهش داده و اجرای برنامه‌ی اصلی را آغاز می‌کند.

۲ مقدمه و فروض مسئله

هدف از این پروژه، طراحی یک سیستم پردازشی کامل با لحاظ کردن محدودیت‌های امنیتی است. فرض بر این است که پردازنده از معماری پایه MIPS (تک‌سیکله) پشتیبانی می‌کند. برای ایجاد یک محیط امن، فروض و نیازمندی‌های زیر در طراحی لحاظ شده‌اند:

- **وضعیت سیستم (Mode_Bit):** سیستم دارای یک رجیستر یک بیتی است. مقدار ۱ نشان‌دهنده حالت امن (Secure) و مقدار ۰ نشان‌دهنده حالت کاربر (User) است. سیستم همواره با ریست شدن در حالت امن راه‌اندازی می‌شود.
- **دستور اختصاصی (DROP_PRIV):** این دستور تنها در حالت امن قابل اجراست و وظیفه آن تغییر مقدار Mode_Bit به صفر (ورود به حالت کاربر) است.
- **نگاشت حافظه (MMIO):** فضای آدرس‌دهی به سه بخش مجزا تقسیم شده است:
 - 0x0000 تا 0x0FFF: متصل به حافظه ROM (محل قرارگیری کد Bootloader)
 - 0x1000 تا 0x1FFF: متصل به حافظه RAM (محل قرارگیری برنامه و داده‌های کاربر)
 - 0x2000 تا 0x20FF: متصل به رجیسترهای شتاب‌دهنده هش سخت‌افزاری
- **قوانین دسترسی:** در حالت User، پردازنده اجازه واکنشی دستورات از ناحیه ROM و همچنین خواندن/نوشتن در ناحیه شتاب‌دهنده هش را ندارد.

۳ نحوه پیاده‌سازی و معماری کلی سیستم

معماری این سیستم از چندین ماژول کلیدی تشکیل شده است که به صورت هماهنگ فرآیند Secure Boot را اجرا می‌کنند. روند کلی کار به این شکل است که پس از ریست شدن، پردازنده در حالت Secure کار خود را از آدرس صفر (ROM) آغاز می‌کند.

کد بوت‌لودر (Bootloader) کلمات حافظه RAM کاربر را خوانده و آن‌ها را به سمت شتاب‌دهنده هش (که بر اساس منطق XOR Accumulator طراحی شده تا سربار سخت‌افزاری کمتری داشته باشد) می‌فرستد. پس از محاسبه هش کل برنامه، بوت‌لودر مقدار نهایی را از شتاب‌دهنده می‌خواند و با یک مقدار مرجع (Reference Hash) مقایسه می‌کند. در صورت تطابق، دستور DROP_PRIV فراخوانی شده و سیستم با پرش به آدرس 0x1000 وارد محیط کاربری می‌شود. برای تضمین امنیت مضاعف، یک ماژول نگهبان سخت‌افزاری (Hardware Watchdog) نیز تعبیه شده است که در صورت نفوذ غیرمجاز یا پرش اشتباه (مثلاً ورود به ناحیه کاربر در حالی که هنوز سیستم در حالت امن است)، پردازنده را به صورت خودکار ریست می‌کند.

۴ نقشه حدودی کار

روند توسعه سیستم در فازهای زیر انجام شده است:

۱. توسعه هسته پردازنده: پیاده‌سازی DataPath و Control Unit با پشتیبانی از دستورات استاندارد MIPS و اضافه کردن منطق Mode_Bit و DROP_PRIV.

۲. طراحی Memory Router: ایجاد ماژول واسط برای هدایت درخواست‌های خواندن/نوشتن پردازنده به RAM، ROM یا ماژول هش بر اساس آدرس و بررسی همزمان سطح دسترسی.

۳. شتاب‌دهنده هش: طراحی ماژول سخت‌افزاری مجزا برای محاسبه سریع هش و اتصال آن به باس MMIO.

۴. نگهبان سخت‌افزاری: پیاده‌سازی Watchdog برای رصد همزمان Program Counter و وضعیت سیستم.

۵. توسعه نرم‌افزار و یکپارچه‌سازی: نوشتن کدهای اسمبلی Bootloader و User Program، اتصال تمامی ماژول‌ها در Top-Level و نوشتن تست‌بنچ‌های جامع برای سناریوهای موفق و

۵ ماژول مسیر داده (DataPath)

۱-۵ توضیح کلی کد

این ماژول، هسته اصلی مسیر داده در پردازنده تک‌سیکله MIPS است که جریان اطلاعات میان واحدها را مدیریت می‌کند. این بخش شامل ثبات شمارنده برنامه (PC)، فایل ثبات‌ها (Register File)، واحد محاسبه و منطق (ALU)، و رجیستر ویژه وضعیت سیستم (Mode_Bit) است. ماژول DataPath سیگنال‌های کنترلی را از Control Unit دریافت کرده و بر اساس آن‌ها، داده‌ها را از حافظه دستورات (Instruction Memory) واکشی کرده، پردازش‌های لازم را انجام داده و نتایج را در ثبات‌ها یا حافظه داده (Data Memory) ذخیره می‌کند.

۲-۵ کاربرد کد در سیستم

کاربرد اصلی این ماژول در این پروژه، علاوه بر اجرای دستورات استاندارد MIPS، مدیریت انتقال امن بین دو حالت Secure و User است. این ماژول وظایف کلیدی زیر را در سیستم بر عهده دارد:

- **اعمال تغییر سطح دسترسی:** با دریافت سیگنال drop_priv_en از واحد کنترل، وضعیت پردازنده به حالت کاربر تغییر می‌کند.
- **هدایت جریان اجرای امن:** هنگام اجرای دستور DROP_PRIV، این ماژول به صورت سخت‌افزاری PC را به آدرس ورود به محیط کاربری (USER_ENTRY_POINT = 0x1000) پرش می‌دهد تا اجرای برنامه‌ی تاییدشده آغاز گردد.
- **توقف اضطراری پردازنده:** در صورت تشخیص نقض امنیتی یا دستور غیرمجاز، شمارنده برنامه متوقف شده (Freeze) و از اجرای کدهای مخرب جلوگیری می‌شود.

۳-۵ کد اصلی

```

1 module MIPS_Datapath (
2     input  wire clk,
3     input  wire rst,
4
5     // Control signals from Control Unit
6     input  wire mem_to_reg,
7     input  wire alu_src,
8     input  wire reg_dst,
9     input  wire reg_write,
10    input  wire jump,
11    input  wire branch,
12    input  wire branch_ne, // 0 = BEQ semantics, 1 = BNE semantics
13    input  wire [2:0] alu_control,
14    input  wire drop_priv_en, // Special signal for DROP_PRIV
15    instruction
16    input  wire illegal_instr, // Illegal instruction flag (
17        combinational)
18
19    input  wire secure_mode_fault, // Security violation flag (
20        combinational)
21
22    input  wire is_lui,
23
24    // Instruction Memory interface
25    input  wire [31:0] instr,
26    output wire [31:0] pc_out,
27
28    // Data Memory / MMIO interface (Memory Router)
29    input  wire [31:0] read_data,
30    output wire [31:0] alu_result,
31    output wire [31:0] write_data,
32
33    // Status outputs
34    output wire mode_bit,

```

```

30     output wire exception_taken, // combinational: illegal_instr |
        secure_mode_fault
31     output wire zero // ALU zero flag, exposed for Control_Unit/Top
32 );
33
34 // --- Internal registers and wires ---
35 reg [31:0] pc;
36 wire [31:0] pc_plus_4;
37 wire [31:0] pc_branch;
38 wire [31:0] pc_next;
39 wire [31:0] pc_jump;
40
41 wire [4:0] write_reg;
42 wire [31:0] result;
43 wire [31:0] src_a;
44 wire [31:0] src_b;
45 wire [31:0] sign_imm;
46 wire [31:0] sign_imm_sh2;
47
48 reg          mode_reg;          // System mode register
49
50 // --- Mode_Bit register implementation ---
51 // On system reset, defaults to secure mode (1)
52 // When drop_priv_en is asserted by control unit, switches to
    user mode (0)
53 always @(posedge clk or posedge rst) begin
54     if (rst) begin
55         mode_reg <= 1'b1; // Secure Mode
56     end else if (drop_priv_en) begin
57         mode_reg <= 1'b0; // User Mode
58     end
59 end

```

```

60  assign mode_bit = mode_reg;
61
62  // --- Exception flag (combinational, single source of truth)
63  // ---
64
65  assign exception_taken = illegal_instr | secure_mode_fault;
66
67  // --- Program Counter (PC) logic ---
68  // Freezes PC in the SAME cycle the fault is decoded (no extra
69  // step taken).
70
71  always @(posedge clk or posedge rst) begin
72      if (rst) begin
73          pc <= 32'b0;
74      end else if (exception_taken) begin
75          // On exception, hold PC. Exception handler redirection
76          // (if any)
77          // is expected to be layered in by the Top module /
78          // handler logic.
79          pc <= pc;
80      end else begin
81          pc <= pc_next;
82      end
83  end
84
85  assign pc_out = pc;
86  assign pc_plus_4 = pc + 4;
87
88  // --- Register File (32x32) ---
89
90  reg [31:0] rf [31:0];
91
92  integer i;
93
94  always @(posedge clk or posedge rst) begin
95      if (rst) begin
96          for (i = 0; i < 32; i = i + 1) rf[i] <= 32'b0;
97      end else if (reg_write && write_reg != 5'b0) begin

```

```

88         rf[write_reg] <= result;
89     end
90 end
91 assign src_a = rf[instr[25:21]]; // rs
92 assign write_data = rf[instr[20:16]]; // rt (sent as write data
    to memory)
93
94 // Destination register selection (rt or rd)
95 assign write_reg = reg_dst ? instr[15:11] : instr[20:16];
96
97 // Sign Extension
98 assign sign_imm = {{16{instr[15]}}, instr[15:0]};
99
100 // --- ALU (Arithmetic Logic Unit) ---
101 assign src_b = alu_src ? sign_imm : write_data;
102
103 reg [31:0] alu_out_reg;
104 always @(*) begin
105     case (alu_control)
106         3'b000: alu_out_reg = src_a & src_b;           // AND
107         3'b001: alu_out_reg = src_a | src_b;           // OR
108         3'b010: alu_out_reg = src_a + src_b;           // ADD
109         3'b011: alu_out_reg = src_a ^ src_b;           // XOR
110         3'b110: alu_out_reg = src_a - src_b;           // SUB
111         3'b111: alu_out_reg = (src_a < src_b) ? 1 : 0; // SLT
112         default: alu_out_reg = 32'b0;
113     endcase
114 end
115 assign alu_result = alu_out_reg;
116 assign zero = (alu_result == 32'b0);
117
118 // --- Branch and Jump logic ---

```

```

119 assign sign_imm_sh2 = {sign_imm[29:0], 2'b00};
120 assign pc_branch = pc_plus_4 + sign_imm_sh2;
121
122 // BEQ: taken when zero==1. BNE: taken when zero==0.
123 wire branch_condition = branch_ne ? ~zero : zero;
124 wire pc_src = branch & branch_condition;
125
126 wire [31:0] pc_mux_branch = pc_src ? pc_branch : pc_plus_4;
127
128 //assign pc_jump = {pc_plus_4[31:28], instr[25:0], 2'b00};
129 // assign pc_next = jump ? pc_jump : pc_mux_branch;
130 localparam USER_ENTRY_POINT = 32'h00001000;
131
132 assign pc_jump = {pc_plus_4[31:28], instr[25:0], 2'b00};
133 assign pc_next = drop_priv_en ? USER_ENTRY_POINT :
134                 jump           ? pc_jump :
135                 pc_mux_branch;
136
137 // --- Data write-back to register file ---
138 wire [31:0] lui_result = {instr[15:0], 16'b0};
139 assign result = is_lui ? lui_result : (mem_to_reg ? read_data :
140                 alu_result);
141
142 endmodule

```

۴-۵ بررسی کد ارائه شده

کد DataPath نوشته شده، با جزئیات کامل تمامی نیازمندی‌های پایه و امنیتی پردازنده را پیاده‌سازی کرده است:

- رجیستر وضعیت امنیتی (mode_reg): با دریافت سیگنال rst (راه‌اندازی مجدد)، این

ثبات مقدار ۱ می‌گیرد تا سیستم در حالت Secure کار خود را آغاز کند. تنها راه خروج از این حالت، فعال شدن سیگنال `drop_priv_en` است.

- **مدیریت استثنا (Exception Handling):** سیگنال `exception_taken` ترکیبی از `illegal_instr`

(دستور غیرمعتبر) و `secure_mode_fault` (نقض قوانین دسترسی) است. بلوک کنترل PC به گونه‌ای طراحی شده که در صورت یک شدن این سیگنال، PC در همان سیکل فریز (Freeze) شده و پردازنده متوقف می‌شود. این یک لایه دفاعی مهم برای جلوگیری از رفتار پیش‌بینی نشده است.

- **فایل ثبات‌ها (Register File):** شامل ۳۲ ثبات ۳۲-بیتی است. شرط `write_reg != 0` تضمین می‌کند که رجیستر شماره صفر (`$zero`) همواره مقدار صفر خود را حفظ کند (فقط - خواندنی).

- **انشعاب و شرط‌ها (Branch & Jump):** با استفاده از سیگنال `branch_ne`، منطق انشعاب می‌تواند دستور BNE (انشعاب در صورت عدم برابری) و BEQ (انشعاب در صورت برابری) را به درستی با استفاده از خروجی `zero` واحد ALU مدیریت کند.

- **پرش سخت‌افزاری DROP_PRIV:** در محاسبه‌ی `pc_next` (آدرس بعدی اجرای برنامه)، بالاترین اولویت به سیگنال `drop_priv_en` داده شده است. در صورت اجرای این دستور، PC مستقیماً به آدرس `0x00001000` (همان `USER_ENTRY_POINT`) که محل شروع RAM کاربری است، پرش می‌کند.

- **پشتیبانی از LUI:** با استفاده از سیگنال `is_lui`، مسیری مجزا برای انتقال داده در نظر گرفته شده است تا بخش پایین Immediate دستور مستقیماً به بخش بالای رجیستر مقصد نوشته شود.

۶ پیاده‌سازی mode_bit

۱-۶ تعریف

mode_bit یک ثبات یک‌بیتی در پردازنده است که وضعیت امنیتی سیستم را مشخص می‌کند:

• مقدار ۱ = حالت امن (Secure Mode)

• مقدار ۰ = حالت کاربر (User Mode)

۲-۶ پیاده‌سازی در DataPath

در ماژول MIPS_Datapath، یک رجیستر داخلی به نام mode_reg تعریف شده است:

```
1 reg mode_reg;           // System mode register
```

و در بخش پورت‌های خروجی ماژول:

```
1 output wire mode_bit,
```

بلوک always زیر مسئول مقداردهی و تغییر mode_bit است:

```
1 always @(posedge clk or posedge rst) begin
2     if (rst)
3         mode_reg <= 1'b1;           // Secure Mode
4     else if (drop_priv_en)
5         mode_reg <= 1'b0;           // User Mode
6 end
7
8 assign mode_bit = mode_reg;
```

نکات پیاده‌سازی:

- با فعال شدن سیگنال rst، مقدار mode_reg روی ۱ قرار می‌گیرد. این تضمین می‌کند که سیستم همیشه در حالت امن راه‌اندازی می‌شود.

- تنها در صورت فعال شدن سیگنال drop_priv_en، مقدار به 0 تغییر می‌کند. این سیگنال توسط واحد کنترل و پس از اجرای موفق دستور DROP_PRIV در حالت امن تولید می‌شود.
- شرط rst بر drop_priv_en اولویت دارد.
- تغییر mode_bit یک فرآیند یک‌طرفه است و تا ریست بعدی قابل بازگشت نیست.

۳-۶ تولید سیگنال drop_priv_en در واحد کنترل

در ماژول Control_Unit، سیگنال drop_priv_en بر اساس دستور DROP_PRIV و مقدار mode_bit تولید می‌شود:

```

1 OP_DROP_PRIV: begin
2     if (mode_bit == 1'b1) begin
3         drop_priv_en = 1'b1;
4     end else begin
5         illegal_instr = 1'b1;
6         secure_mode_fault = 1'b1;
7         exception_pc = pc;
8         exception_taken = 1'b1;
9     end
10 end

```

اگر $mode_bit = 1$ (حالت امن)، دستور DROP_PRIV مجاز است و $drop_priv_en = 1$ می‌شود. اگر $mode_bit = 0$ (حالت کاربر)، دستور غیرمجاز است و $secure_mode_fault$ فعال می‌شود.

۴-۶ اتصال mode_bit در ماژول سطح بالا

در ماژول MIPS_Processor_Top، mode_bit از خروجی DataPath دریافت و به سایر ماژول‌ها ارسال می‌شود:

```

1 MIPS_Datapath dp_inst (
2     .mode_bit(mode_bit),    //      DataPath

```

```

3      // ...
4  );
5
6  Control_Unit cu_inst (
7      .mode_bit(mode_bit),    //      Control Unit
8      // ...
9  );
10
11  Memory_Router mem_inst (
12      .mode_bit(mode_bit),    //      Memory Router
13      // ...
14  );

```

۵-۶ کنترل دسترسی در روتر حافظه

روتر حافظه از mode_bit برای اعمال سیاست‌های دسترسی استفاده می‌کند:

```

1  assign instr_allow = (mode_bit == 1'b1) ? instr_allow_secure :
    instr_allow_user;
2  assign data_read_allow = (mode_bit == 1'b1) ? data_read_allow_secure
    : data_read_allow_user;
3  assign data_write_allow = (mode_bit == 1'b1) ?
    data_write_allow_secure : data_write_allow_user;

```

سیاست‌های دسترسی:

حالت User	حالت Secure	منطقه حافظه
ممنوع	مجاز	Boot ROM 0x0000-0x0FFF
مجاز	مجاز	User RAM 0x1000-0x1FFF
ممنوع	مجاز	Crypto 0x2000-0x20FF

جدول ۱: سیاست‌های دسترسی بر اساس mode_bit

در حالت Secure، تمامی نواحی حافظه قابل دسترسی هستند. در حالت User، تنها دسترسی به

User RAM مجاز است.

۶-۶ نظارت توسط نگهبان سخت‌افزاری

ماژول Hardware_Watchdog از mode_bit برای تشخیص تخلفات استفاده می‌کند:

```
1 parameter SECURE_MODE = 1'b1;  
2 wire violation = (Mode_Bit == SECURE_MODE) && pc_in_ram;  
3 assign sys_rst_out = rst_in | violation | violation_latched;
```

اگر mode_bit = 1 (حالت امن) و PC در محدوده RAM باشد، سیستم ریست می‌شود.

۷-۶ خلاصه

۱. سیستم با mode_bit = 1 (حالت امن) راه‌اندازی می‌شود.
 ۲. بوت‌لودر در حالت امن اجرا شده و هش را بررسی می‌کند.
 ۳. پس از تأیید هش، دستور DROP_PRIV اجرا می‌شود.
 ۴. mode_bit = 0 شده و سیستم وارد حالت کاربر می‌شود.
 ۵. در صورت وقوع تخلف، Watchdog سیستم را ریست می‌کند.
- تغییر mode_bit یک‌طرفه است و تا ریست بعدی قابل بازگشت نیست.

۷ ماژول واحد کنترل (Control Unit)

۱-۷ توضیح کلی کد

واحد کنترل (Control Unit) به منزله‌ی مغز متفکر پردازنده است. این ماژول کدهای عملیاتی (Opcode) و کدهای تابعی (Funct) را از دستور واکشی شده دریافت کرده و سیگنال‌های کنترلی لازم برای هدایت ماژول مسیر داده (DataPath) را تولید می‌کند. این سیگنال‌ها تعیین می‌کنند که واحد ALU چه عملیاتی انجام دهد، آیا داده‌ای در فایل ثبات‌ها (Register File) یا حافظه نوشته

شود یا خیر، و آیا جریان برنامه نیاز به انشعاب (Branch) یا پرش (Jump) دارد یا به صورت ترتیبی ادامه می‌یابد.

۲-۷ کاربرد کد در سیستم

در این سیستم امنیتی، واحد کنترل علاوه بر رمزگشایی دستورات استاندارد MIPS، وظیفه اعمال سیاست‌های دسترسی و نظارت بر امنیت اجرا را نیز بر عهده دارد. کاربردهای کلیدی این ماژول در سیستم عبارتند از:

- **کنترل سطح دسترسی:** پردازش دستور ویژه DROP_PRIV و صدور فرمان تغییر وضعیت امنیتی (drop_priv_en) تنها در صورتی که پردازنده در حالت امن (Secure Mode) باشد.
- **تشخیص تخلف امنیتی:** در صورتی که برنامه در حالت کاربر (User Mode) تلاش کند دستور DROP_PRIV را اجرا کند، واحد کنترل فوراً یک خطای دسترسی (secure_mode_fault) صادر می‌کند.
- **تشخیص دستورات غیرمجاز:** هرگونه آپ‌کد (Opcode) یا کد تابعی (Funct) ناشناخته به عنوان یک دستور غیرقانونی تلقی شده و باعث توقف فوری اجرای برنامه (exception_taken) می‌شود.

۳-۷ کد اصلی

Listing 2: Control_Unit.v

```
1 module Control_Unit (  
2     input  wire [5:0]  opcode ,  
3     input  wire [5:0]  funct ,  
4     input  wire mode_bit ,  
5     input  wire zero ,  
6     input  wire [31:0] pc ,  
7     output reg  mem_to_reg ,  
8     output reg  alu_src ,  
9     output reg  reg_dst ,
```

```

10     output reg    reg_write,
11     output reg    jump,
12     output reg    branch,
13     output reg    branch_ne,
14     output reg [2:0]    alu_control,
15     output reg    drop_priv_en,
16     output reg    illegal_instr,
17     output reg    secure_mode_fault,
18     output reg [31:0]    exception_pc,
19     output reg    exception_taken,
20     output reg    is_lui
21 );
22
23     localparam OP_R_TYPE = 6'b0000000;
24     localparam OP_ADDI = 6'b001000;
25     localparam OP_ORI  = 6'b001101;
26     localparam OP_XORI = 6'b001110;
27     localparam OP_LW   = 6'b100011;
28     localparam OP_SW   = 6'b101011;
29     localparam OP_BEQ  = 6'b000100;
30     localparam OP_BNE  = 6'b000101;
31     localparam OP_J    = 6'b000010;
32     localparam OP_DROP_PRIV  = 6'b111111;
33     localparam OP_LUI = 6'b001111;
34
35     localparam FUNC_ADD = 6'b100000;
36     localparam FUNC_SUB = 6'b100010;
37     localparam FUNC_AND = 6'b100100;
38     localparam FUNC_OR  = 6'b100101;
39     localparam FUNC_XOR = 6'b100110;
40     localparam FUNC_SLT = 6'b101010;
41

```

```

42     localparam ALU_AND = 3'b000;
43     localparam ALU_OR  = 3'b001;
44     localparam ALU_ADD = 3'b010;
45     localparam ALU_XOR = 3'b011;
46     localparam ALU_SUB = 3'b110;
47     localparam ALU_SLT = 3'b111;
48
49     always @(*) begin
50         mem_to_reg = 1'b0;
51         alu_src = 1'b0;
52         reg_dst = 1'b0;
53         reg_write = 1'b0;
54         jump = 1'b0;
55         branch = 1'b0;
56         branch_ne = 1'b0;
57         alu_control = ALU_ADD;
58         drop_priv_en = 1'b0;
59         illegal_instr = 1'b0;
60         secure_mode_fault = 1'b0;
61         exception_pc = 32'b0;
62         exception_taken = 1'b0;
63         is_lui = 1'b0;
64
65         case (opcode)
66             OP_R_TYPE: begin
67                 reg_dst    = 1'b1;
68                 reg_write = 1'b1;
69                 alu_src    = 1'b0;
70                 mem_to_reg = 1'b0;
71                 case (funct)
72                     FUNC_ADD: alu_control = ALU_ADD;
73                     FUNC_SUB: alu_control = ALU_SUB;

```

```

74         FUNC_AND: alu_control = ALU_AND;
75         FUNC_OR:  alu_control = ALU_OR;
76         FUNC_XOR: alu_control = ALU_XOR;
77         FUNC_SLT: alu_control = ALU_SLT;
78         default: begin
79             illegal_instr = 1'b1;
80             reg_write = 1'b0;
81             exception_pc = pc;
82             exception_taken = 1'b1;
83         end
84     endcase
85 end
86 OP_ADDI: begin
87     alu_src      = 1'b1;
88     reg_write    = 1'b1;
89     mem_to_reg   = 1'b0;
90     alu_control  = ALU_ADD;
91 end
92 OP_ORI: begin
93     alu_src      = 1'b1;
94     reg_write    = 1'b1;
95     mem_to_reg   = 1'b0;
96     alu_control  = ALU_OR;
97 end
98 OP_XORI: begin
99     alu_src      = 1'b1;
100    reg_write    = 1'b1;
101    mem_to_reg   = 1'b0;
102    alu_control  = ALU_XOR;
103 end
104 OP_LW: begin
105     alu_src      = 1'b1;

```

```

106         reg_write    = 1'b1;
107         mem_to_reg    = 1'b1;
108         alu_control    = ALU_ADD;
109     end
110     OP_SW: begin
111         alu_src        = 1'b1;
112         reg_write      = 1'b0;
113         mem_to_reg     = 1'b0;
114         alu_control    = ALU_ADD;
115     end
116     OP_BEQ: begin
117         branch         = 1'b1;
118         branch_ne      = 1'b0;
119         alu_control    = ALU_SUB;
120     end
121     OP_BNE: begin
122         branch         = 1'b1;
123         branch_ne      = 1'b1;
124         alu_control    = ALU_SUB;
125     end
126     OP_J: begin
127         jump           = 1'b1;
128     end
129     OP_DROP_PRIV: begin
130         if (mode_bit == 1'b1) begin
131             drop_priv_en = 1'b1;
132         end else begin
133             illegal_instr    = 1'b1;
134             secure_mode_fault = 1'b1;
135             exception_pc     = pc;
136             exception_taken  = 1'b1;
137         end

```

```

138         end
139     OP_LUI: begin                // <-- NEW CASE
140         alu_src      = 1'b1; // immediate is used
141         reg_write    = 1'b1;
142         mem_to_reg   = 1'b0;
143         alu_control  = ALU_ADD; // not used
144         is_lui       = 1'b1;
145     end
146     default: begin
147         illegal_instr = 1'b1;
148         exception_pc  = pc;
149         exception_taken = 1'b1;
150     end
151 endcase
152 end
153 endmodule

```

۴-۷ بررسی کد ارائه شده

کد واحد کنترل از یک بلوک ترکیبی (Combinational Block) تشکیل شده است که با استفاده از ساختار Case، سیگنال‌ها را بر اساس دستور فعلی مقداردهی می‌کند. جزئیات عملکرد این بخش به شرح زیر است:

- **جلوگیری از ایجاد لچ (Latch):** در ابتدای بلوک (*@always، تمامی خروجی‌ها با مقادیر پیش فرض امن (اکثراً صفر) مقداردهی اولیه می‌شوند. این تکنیک طراحی تضمین می‌کند که در هیچ شرایطی مدار ترکیبی مقادیر قبلی خود را حفظ نکرده و از تولید سخت افزار ناخواسته جلوگیری می‌شود.

- **رمزگشایی دستورات استاندارد:**

– در دستورات R-Type، سیگنال‌های reg_dst و reg_write یک می‌شوند و نوع عملیات با استفاده از متغیر funct برای ALU تعیین می‌شود. در صورت ناشناخته بودن funct،

سیستم با فعال کردن `illegal_instr` و `exception_taken` وارد حالت خطا می‌شود.

– دستورات با عملوند فوری (Immediate) مانند `ADDI`، `ORI` و `XORI` با یک کردن `alu_src`، داده دوم `ALU` را از بخش پایین دستور تامین می‌کنند.

– دستورات دسترسی به حافظه (`LW` و `SW`) و انشعاب‌ها (`BEQ` و `BNE`) نیز سیگنال‌های مرتبط با حافظه و پرش شرطی را مدیریت می‌کنند. دستور `LUI` نیز با تولید سیگنال اختصاصی `is_lui` پشتیبانی شده است.

• **منطق دستور امنیتی `DROP_PRIV`:** با رسیدن آپ‌کد اختصاصی این دستور (`6'b111111`)، واحد کنترل وضعیت فعلی پردازنده را از طریق ورودی `mode_bit` بررسی می‌کند. اگر `mode_bit == 1` باشد (حالت امن)، مجوز کاهش دسترسی (`drop_priv_en = 1`) صادر می‌شود. اما اگر این دستور در حالت کاربر اجرا شود، واحد کنترل با فعال‌سازی متغیرهای `illegal_instr`، `secure_mode_fault` و `exception_taken` و همچنین ذخیره آدرس خطا کار در `exception_pc`، پردازنده را از اجرای آن منع می‌کند.

• **مدیریت حالت پیش‌فرض (Default Case):** هر آپ‌کدی که در لیست مجاز تعریف نشده باشد مستقیماً به بخش `default` رفته و به عنوان استثنا و دستور غیرمجاز (`Illegal Instruction`) ثبت می‌گردد.

۸ ماژول روتر حافظه (Memory Router)

۱-۸ توضیح کلی کد

ماژول روتر حافظه (Memory Router) به عنوان مدیر و واسط ارتباطی بین هسته پردازنده و نواحی مختلف حافظه عمل می‌کند. این واحد وظیفه دارد درخواست‌های خواندن و نوشتن حافظه دستورات (Instruction Memory) و حافظه داده (Data Memory) را دریافت کرده و بر اساس آدرس درخواستی (Address Decoding)، آن‌ها را به بخش مناسب سیستم هدایت کند. همچنین با استفاده از نگاشت حافظه (Memory Mapped I/O)، دستگاه‌های جانبی نظیر شتاب‌دهنده هش در کنار حافظه‌های RAM و ROM در یک فضای آدرس‌دهی یکپارچه قرار گرفته‌اند.

۲-۸ کاربرد کد در سیستم

این ماژول نقش اساسی در تامین امنیت سیستم ایفا می‌کند. کاربردهای اصلی آن عبارتند از:

- **جداسازی نواحی حافظه:** تعریف سه ناحیه مجزا برای بوت‌لودر (Boot ROM)، برنامه کاربر (User RAM) و دستگاه‌های جانبی (Crypto/Hash).

- **اعمال کنترل دسترسی سخت‌افزاری:** با استفاده از سیگنال mode_bit، این ماژول دسترسی برنامه‌های کاربر را به نواحی حساس (مانند واکشی دستور از ROM یا ارتباط با ماژول هش) مسدود می‌کند.

- **تشخیص تخلفات:** در صورت تلاش پردازنده برای دسترسی غیرمجاز، سیگنال violation_detected صادر شده که برای متوقف کردن سیستم یا راه‌اندازی ماژول نگهبان (Watchdog) استفاده می‌شود.

۳-۸ کد اصلی

Listing 3: Memory_Router.v

```
1 module Memory_Router (  
2     input  wire clk,  
3     input  wire rst,
```

```

4      input  wire mode_bit,
5      input  wire secure_mode_fault,
6      input  wire exception_taken,
7
8      input  wire instr_read_en,
9      input  wire [31:0] instr_addr,
10     output wire instr_allow,
11     output wire [31:0] instr_read_data,
12     output wire instr_violation,
13
14     input  wire data_read_en,
15     input  wire [31:0] data_read_addr,
16     output wire data_read_allow,
17     output wire [31:0] data_read_data,
18
19     input  wire data_write_en,
20     input  wire [31:0] data_write_addr,
21     input  wire [31:0] data_write_data,
22     output wire data_write_allow,
23
24     output wire violation_detected
25 );
26
27 // Memory Map Definitions
28 localparam BOOT_ROM_BASE = 32'h00000000;
29 localparam BOOT_ROM_END = 32'h00000FFF;
30 localparam USER_RAM_BASE = 32'h00001000;
31 localparam USER_RAM_END = 32'h00001FFF;
32 localparam CRYPTO_BASE = 32'h00002000;
33 localparam CRYPTO_END = 32'h000020FF;
34

```

```

35 parameter BOOT_ROM_FILE = "C:/Users/Asus/Downloads/
    CA_Project_kharab_watchdog/CA_Project/boot_rom.hex";
36 // Memory arrays
37 reg [31:0] boot_rom [0:1023];
38 reg [31:0] user_ram [0:1023];
39
40 initial begin
41     $readmemh(BOOT_ROM_FILE, boot_rom);
42 end
43
44 wire crypto_cs;
45 wire crypto_we;
46 wire [31:0] crypto_read_data;
47
48 assign crypto_cs = (data_read_en && (data_read_addr >=
    CRYPTO_BASE && data_read_addr <= CRYPTO_END)) || (
    data_write_en && (data_write_addr >= CRYPTO_BASE &&
    data_write_addr <= CRYPTO_END));
49
50 assign crypto_we = data_write_en && (data_write_addr >=
    CRYPTO_BASE && data_write_addr <= CRYPTO_END);
51
52 Hash_Accelerator hash_unit (
53     .clk(clk),
54     .rst(rst),
55     .cs(crypto_cs),
56     .we(crypto_we),
57     .addr(data_write_en ? data_write_addr : data_read_addr),
58     .write_data(data_write_data),
59     .read_data(crypto_read_data)
60 );
61

```

```

62 wire instr_in_boot_rom = (instr_addr >= BOOT_ROM_BASE &&
    instr_addr <= BOOT_ROM_END);
63 wire instr_in_user_ram = (instr_addr >= USER_RAM_BASE &&
    instr_addr <= USER_RAM_END);
64 wire instr_in_crypto    = (instr_addr >= CRYPTO_BASE &&
    instr_addr <= CRYPTO_END);
65
66 wire data_read_in_boot_rom = (data_read_addr >= BOOT_ROM_BASE &&
    data_read_addr <= BOOT_ROM_END);
67 wire data_read_in_user_ram = (data_read_addr >= USER_RAM_BASE &&
    data_read_addr <= USER_RAM_END);
68 wire data_read_in_crypto    = (data_read_addr >= CRYPTO_BASE &&
    data_read_addr <= CRYPTO_END);
69
70 wire data_write_in_user_ram = (data_write_addr >= USER_RAM_BASE
    && data_write_addr <= USER_RAM_END);
71 wire data_write_in_crypto    = (data_write_addr >= CRYPTO_BASE &&
    data_write_addr <= CRYPTO_END);
72
73 wire instr_allow_secure = (instr_in_boot_rom ||
    instr_in_user_ram || instr_in_crypto);
74 wire instr_allow_user    = (instr_in_user_ram);
75
76 assign instr_allow = (mode_bit == 1'b1) ? instr_allow_secure :
    instr_allow_user;
77
78 wire data_read_allow_secure = (data_read_in_user_ram ||
    data_read_in_crypto);
79 wire data_read_allow_user    = (data_read_in_user_ram);
80
81 assign data_read_allow = (mode_bit == 1'b1) ?
    data_read_allow_secure : data_read_allow_user;

```

```

82
83 wire data_write_allow_secure = (data_write_in_user_ram ||
    data_write_in_crypto);
84 wire data_write_allow_user    = (data_write_in_user_ram);
85
86 assign data_write_allow = (mode_bit == 1'b1) ?
    data_write_allow_secure : data_write_allow_user;
87
88 wire instr_violation_wire = instr_read_en && !instr_allow;
89 wire data_read_violation  = data_read_en && !data_read_allow;
90 wire data_write_violation = data_write_en && !data_write_allow;
91
92 assign instr_violation = instr_violation_wire;
93
94 reg violation_detected_reg;
95 assign violation_detected = violation_detected_reg;
96
97 always @(posedge clk or posedge rst) begin
98     if (rst) begin
99         violation_detected_reg <= 1'b0;
100     end else if (instr_violation_wire || data_read_violation ||
        data_write_violation || secure_mode_fault) begin
101         violation_detected_reg <= 1'b1;
102     end else begin
103         violation_detected_reg <= 1'b0;
104     end
105 end
106
107
108 // Instruction Memory -- COMBINATIONAL READ
109 assign instr_read_data =
110     (instr_read_en && instr_allow) ?

```

```

111         (instr_in_boot_rom ? boot_rom[instr_addr[11:2]] :
112         instr_in_user_ram ? user_ram[instr_addr[11:2]] :
113         32'b0)
114     : 32'b0;
115
116     // Data Read Memory -- COMBINATIONAL READ
117     assign data_read_data =
118         (data_read_en && data_read_allow) ?
119         (data_read_in_boot_rom ? boot_rom[data_read_addr[11:2]]
120         :
121         data_read_in_user_ram ? user_ram[data_read_addr[11:2]]
122         :
123         data_read_in_crypto ? crypto_read_data : 32'b0)
124     : 32'b0;
125
126     // Data Write Memory -- SYNCHRONOUS
127     // CRITICAL FIX: Only reset user_ram, NOT boot_rom!
128     integer i;
129     always @(posedge clk or posedge rst) begin
130         if (rst) begin
131             // Clear ONLY the writable RAM
132             for (i = 0; i < 1024; i = i + 1) begin
133                 user_ram[i] <= 32'b0;
134             end
135             // BOOT ROM IS NOT CLEARED - it retains its programmed
136             data
137             // (The initial block loaded it, and we never overwrite
138             it)
139         end else if (data_write_en && data_write_allow) begin
140             if (data_write_in_user_ram) begin
141                 user_ram[data_write_addr[11:2]] <= data_write_data;
142             end
143         end
144     end

```

```

139         // Writes to crypto are handled by Hash_Accelerator
140         module
141             end
142         end
143     endmodule

```

۴-۸ بررسی کد ارائه شده

کد روتر حافظه دارای چندین بخش کلیدی است که با دقت برای الزامات امنیتی تنظیم شده‌اند:

- **تعریف نگاشت حافظه (Memory Map):** آدرس‌های پایه و پایانی برای BOOT_ROM، USER_RAM و CRYPTO_BASE با استفاده از localparam تعریف شده‌اند. این مقادیر دقیقاً با فرضیات پروژه تطابق دارند.
- **راه‌اندازی حافظه ROM:** از طریق بلوک initial و دستور \$readmemh، کد اجرایی از یک فایل Hex خارجی مستقیماً به داخل حافظه boot_rom بارگذاری می‌شود.
- **یکپارچگی شتاب‌دهنده هش:** ماژول Hash_Accelerator در داخل همین روتر فراخوانی (Instantiate) شده است. اگر آدرس درخواستی مربوط به رنج CRYPTO باشد، سیگنال Chip Select (cs) فعال شده و داده‌ها مستقیماً از/به این ماژول هدایت می‌شوند.
- **سطوح دسترسی (Permissions):** برای هر سه عمل واکنشی دستور (Instruction Fetch)، خواندن داده و نوشتن داده، منطق ترکیبی مجزایی بر اساس mode_bit پیاده‌سازی شده است. در حالت امن (secure) تمامی دسترسی‌های مجاز در این پروژه باز هستند، اما در حالت کاربر (user)، متغیرهای __allow_user محدود شده و تنها اجازه دسترسی به USER_RAM را می‌دهند.
- **ثبت تخلف (Violation Detection):** در صورت تلاش برای دسترسی به ناحیه‌ای که مجوز آن صادر نشده است (!allow)، ثبات سنکرون violation_detected_reg یک شده و به سیستم هشدار می‌دهد.
- **حفاظت از محتوای ROM:** در بلوک always مربوط به نوشتن داده که با کلاک همگام (Synchronous) است، یک نکته حیاتی رعایت شده است: در زمان وقوع سیگنال ریست

(rst)، تنها حافظه user_ram پاک (Clear) می‌شود و محتوای boot_rom برای راه‌اندازی مجدد دست‌نخورده باقی می‌ماند.

۹ ماژول شتاب‌دهنده هش (Hash Accelerator)

۹-۱ توضیح کلی کد

این ماژول یک شتاب‌دهنده سخت‌افزاری برای انجام عملیات هش (Hash) مبتنی بر عملگر XOR است. ارتباط با این واحد از طریق رابط نگاشت حافظه (Memory-Mapped I/O) انجام می‌پذیرد. شتاب‌دهنده هش دارای سه ثبت (Register) اصلی است: ثبت کنترلی (CTRL) برای ریست کردن وضعیت، ثبت داده (DATA) برای دریافت داده‌های جدید، و ثبت نتیجه (HASH) برای خواندن مقدار نهایی. هسته این ماژول یک انبار (Accumulator) است که داده‌های ورودی را به صورت متوالی با یکدیگر XOR می‌کند.

۹-۲ کاربرد کد در سیستم

در معماری این پروژه، شتاب‌دهنده هش نقش کلیدی در فرآیند بوت امن (Secure Boot) و تأیید یکپارچگی (Integrity) داده‌ها دارد. کاربردهای اصلی آن عبارتند از:

- **تسریع عملیات رمزنگاری:** محاسبه هش به صورت سخت‌افزاری در یک چرخه کلاک (Clock Cycle) انجام می‌شود که بار پردازشی هسته پردازنده (CPU) را به شدت کاهش می‌دهد.
- **اعتبارسنجی نرم‌افزار:** پیش از اجرای برنامه‌های کاربر، سیستم عامل یا بوت‌لودر می‌تواند بلوک‌های حافظه را خوانده و به این ماژول ارسال کند تا از دست‌نخورده بودن کدهای اجرایی اطمینان حاصل شود.
- **طراحی ماژولار مبتنی بر MMIO:** به واسطه استفاده از این شتاب‌دهنده، پردازنده نیازی به دستورات اختصاصی جدید برای عملیات هش ندارد و تنها با دستورات استاندارد دسترسی به حافظه (SW و LW) با آن تعامل می‌کند.

۹-۳ کد اصلی

Listing 4: Hash_Accelerator.v

```
1 module Hash_Accelerator (
```

```

2      input  wire clk,
3      input  wire rst,
4
5      // Memory-Mapped I/O Interface
6      input  wire cs,
7      input  wire we,
8      input  wire [31:0] addr,
9      input  wire [31:0] write_data,
10     output reg [31:0] read_data
11 );
12
13     // Register Offsets
14     localparam CTRL_REG = 8'h00; // Offset 0x00 -> 0x2000
15     localparam DATA_REG = 8'h04; // Offset 0x04 -> 0x2004
16     localparam HASH_REG = 8'h08; // Offset 0x08 -> 0x2008
17
18     // Internal State: The Accumulator
19     reg [31:0] hash_acc;
20
21     always @(posedge clk or posedge rst) begin
22         if (rst) begin
23             hash_acc <= 32'b0;
24         end else if (cs && we) begin
25             if (addr[7:0] == CTRL_REG) begin
26                 // Reset accumulator if bit 0 is written as 1
27                 if (write_data[0] == 1'b1) begin
28                     hash_acc <= 32'b0;
29                 end
30             end else if (addr[7:0] == DATA_REG) begin
31                 // XOR new data with current hash
32                 hash_acc <= hash_acc ^ write_data;
33             end

```

```

34         end
35     end
36
37     always @(*) begin
38         read_data = 32'b0; // Default
39         if (cs && !we) begin
40             case (addr[7:0])
41                 HASH_REG: read_data = hash_acc;
42                 CTRL_REG: read_data = 32'b0; // Status/Control (
43                             optional to read)
44                 default: read_data = 32'b0;
45             endcase
46         end
47     end
48 endmodule

```

۴-۹ بررسی کد ارائه شده

کد این ماژول با رعایت اصول طراحی همگام (Synchronous Design) نوشته شده است و مکانیزم‌های آن به شرح زیر تحلیل می‌شوند:

- **تعریف آفست‌ها (Register Offsets):** با استفاده از localparam، سه آدرس ۸ بیتی به عنوان آفست تعریف شده‌اند که به ترتیب شامل 0x00 برای کنترل، 0x04 برای داده و 0x08 برای هش می‌باشند. این آدرس‌ها در روتر حافظه با آدرس پایه 0x2000 جمع می‌شوند.
 - **منطق نوشتن (بلوک Sequential):** در بلوک always همگام با لبه بالا رونده کلاک، در صورتی که سیگنال انتخاب تراشه (cs) و فعال‌ساز نوشتن (we) یک باشند:
- اگر آدرس برابر با CTRL_REG باشد و بیت صفرم داده ورودی (write_data[0]) یک باشد، انباره داخلی (hash_acc) صفر می‌شود. این ویژگی برای راه‌اندازی محاسبه یک هش جدید ضروری است.

– اگر آدرس برابر با DATA_REG باشد، داده ورودی با مقدار فعلی hash_acc به صورت بیت به بیت XOR شده و نتیجه مجدداً در انباره ذخیره می شود.

- **منطق خواندن (بلوک Combinational):** در بلوک (*) @always، وقتی پردازنده درخواست خواندن دارد ($cs = 1$ و $we = 0$)، ماژول با توجه به آدرس درخواستی مقدار خروجی read_data را تعیین می کند. خواندن از آدرس HASH_REG مقدار نهایی محاسبه شده در hash_acc را روی گذرگاه داده قرار می دهد. برای سایر آدرس ها یا در حالت پیش فرض، خروجی صفر خواهد بود تا از نشت اطلاعات ناخواسته جلوگیری شود.

۱۰ ماژول نگهبان سخت‌افزاری (Hardware Watchdog)

۱-۱۰ توضیح کلی کد

این ماژول یک نگهبان سخت‌افزاری (Hardware Watchdog) است که وظیفه نظارت پیوسته بر شمارنده برنامه (Program Counter یا PC) و بیت وضعیت پردازنده (Mode_Bit) را بر عهده دارد. هدف اصلی این ماژول، تشخیص خطاهای امنیتی بحرانی در سطح سخت‌افزار و اعمال ریست (Reset) فوری به کل سیستم در صورت بروز هرگونه رفتار غیرمجاز در مسیر اجرای دستورات است.

۲-۱۰ کاربرد کد در سیستم

در معماری امن طراحی شده، کدهای حالت امن (Secure Mode) که دارای بالاترین سطح دسترسی هستند، تحت هیچ شرایطی نباید از حافظه اختصاصی کاربر (User RAM) اجرا شوند؛ زیرا این امر می‌تواند منجر به اجرای کدهای مخرب تزریق‌شده توسط کاربر شود. این ماژول به عنوان یک سد دفاعی فیزیکی عمل کرده و امنیت سیستم را تضمین می‌کند. کاربردهای اصلی آن عبارتند از:

- **جلوگیری از اجرای کدهای مخرب:** شناسایی سریع اجرای کد در حالت Secure از آدرس‌های نامعتبر (حافظه کاربر).
- **ریست بدون تأخیر (Zero-Latency Reset):** توقف فوری پردازنده پیش از آنکه دستور غیرمجاز بتواند وضعیت سیستم یا حافظه را تغییر دهد.
- **قفل شدن در حالت امن:** با استفاده از مکانیزم قفل‌شونده (Latch)، سیستم تا زمان دریافت سیگنال ریست معتبر خارجی در حالت ریست باقی می‌ماند.

۳-۱۰ کد اصلی

Listing 5: Hardware_Watchdog.v

```
1 module Hardware_Watchdog (  
2     input wire clk,  
3     input wire rst_in,  
4     input wire [31:0] PC,
```

```

5    input wire Mode_Bit,
6
7    output wire sys_rst_out
8 );
9
10   parameter RAM_START_ADDR = 32'h0000_1000;
11   parameter RAM_END_ADDR   = 32'h0000_1FFF;
12   parameter SECURE_MODE    = 1'b1;
13
14   wire pc_in_ram = (PC >= RAM_START_ADDR) && (PC <= RAM_END_ADDR);
15
16   wire violation = (Mode_Bit == SECURE_MODE) && pc_in_ram;
17
18   reg violation_latched;
19   always @(posedge clk or posedge rst_in) begin
20       if (rst_in)
21           violation_latched <= 1'b0;
22       else if (violation)
23           violation_latched <= 1'b1;
24   end
25
26   // External reset is combinational/pass-through -> no added
27   // latency,
28   // deasserts in the exact same instant rst_in does.
29   assign sys_rst_out = rst_in | violation | violation_latched;
30 endmodule

```

۴-۱۰ بررسی کد ارائه شده

کد این ماژول با رویکرد نظارت ترکیبی (Combinational) و ترتیبی (Sequential) طراحی شده است:

- **تعریف محدوده‌ها (Parameters):** با استفاده از کلمه کلیدی parameter، آدرس شروع (0x1000) و پایان (0x1FFF) حافظه کاربر (RAM) و همچنین مقدار منطقی حالت امن (SECURE_MODE = 1) تعریف شده‌اند که خوانایی کد را به شدت افزایش می‌دهد.

- **تشخیص نقض امنیتی (Violation Detection):** سیگنال pc_in_ram به صورت ترکیبی بررسی می‌کند که آیا PC در محدوده حافظه کاربر قرار دارد یا خیر. سپس، سیگنال violation در صورتی که پردازنده در حالت امن باشد و همزمان PC در محدوده RAM کاربر باشد، برابر با یک منطقی می‌شود.

- **ثبت وضعیت خطا (Violation Latching):** در بلوک always همگام با کلاک، در صورت فعال شدن سیگنال violation، فلیپ‌فلاپ violation_latched یک شده و وضعیت خطا را در خود ذخیره می‌کند. این مقدار تنها در صورت اعمال ریست خارجی (rst_in) پاک می‌شود.

- **تولید خروجی ریست (sys_rst_out):** سیگنال ریست نهایی سیستم ترکیبی از عملگر منطقی OR بین ریست خارجی، خطای لحظه‌ای و خطای ثبت شده است. استفاده از تخصیص پیوسته (assign) باعث می‌شود که ریست بدون هیچ‌گونه تأخیر کلاک (Zero-Latency) اعمال گردد.

۱۱ ماژول سطح بالا (MIPS_Processor_Top)

۱-۱۱ توضیح کلی کد

این ماژول، بالاترین سطح سلسله‌مراتب (Top-Level) در طراحی پردازنده است که تمامی زیرماژول‌های مجزا شامل مسیر داده (Datapath)، واحد کنترل (Control Unit)، روتر حافظه (Memory Router) و نگهبان سخت‌افزاری (Hardware Watchdog) را در کنار یکدیگر تجمیع می‌کند. در این ماژول هیچ منطق ترتیبی (Sequential) جدیدی تعریف نمی‌شود، بلکه تنها وظیفه سیم‌کشی (Wiring) و اتصال پورت‌های ورودی و خروجی ماژول‌های مختلف به یکدیگر بر عهده آن است.

۲-۱۱ کاربرد کد در سیستم

این ماژول به عنوان هسته مرکزی سیستم عمل کرده و ارتباطات بین بخش‌های مختلف پردازنده را یکپارچه می‌سازد. کاربردهای کلیدی آن در سیستم عبارتند از:

- یکپارچه‌سازی ماژول‌ها: برقراری ارتباط صحیح بین واحد کنترل (جهت تولید سیگنال‌های کنترلی) و مسیر داده (جهت اجرای دستورات).
- مدیریت یکپارچه ریست (Reset): تجمیع سیگنال ریست خارجی (rst) و ریست تولیدشده توسط نگهبان سخت‌افزاری (watchdog_reset) برای اعمال یک ریست سراسری (system_reset) به کل سیستم در شرایط بحرانی.
- ارتباط پردازنده و حافظه: مدیریت سیگنال‌های خواندن و نوشتن (Read/Write Enables) و اتصال گذرگاه‌های آدرس و داده پردازنده به روتر حافظه.

۳-۱۱ کد اصلی

Listing 6: MIPS_Processor_Top.v

```
1 module MIPS_Processor_Top (  
2     input  wire clk,  
3     input  wire rst,  
4     output wire [31:0] pc_out,
```

```

5      output wire mode_bit,
6      output wire exception_taken
7  );
8
9      wire watchdog_reset;
10
11     wire system_reset = rst | watchdog_reset;
12
13
14     wire mem_to_reg;
15     wire alu_src;
16     wire reg_dst;
17     wire reg_write;
18     wire jump;
19     wire branch;
20     wire branch_ne;
21     wire [2:0] alu_control;
22     wire drop_priv_en;
23     wire illegal_instr;
24     wire secure_mode_fault;
25     wire [31:0] exception_pc;
26     wire cu_exception_taken;
27     wire is_lui;
28
29
30     wire [31:0] pc;
31     wire [31:0] instr;
32     wire zero;
33
34     wire [31:0] alu_result;
35     wire [31:0] read_data;
36     wire [31:0] write_data;
37
38     wire dp_exception_taken;    // single source of truth, drives
39     output

```

```

36  wire instr_allow;
37  wire instr_violation;
38  wire data_read_allow;
39  wire data_write_allow;
40  wire mem_violation_detected;
41
42  // Instruction fetch is always "enabled" every cycle in a single
    -cycle design
43  wire          instr_read_en = 1'b1;
44
45  // Data memory is accessed for LW (read) or SW (write); derive
    enables
46  // from the same control signals the datapath uses.
47  wire data_read_en  = mem_to_reg;           // LW asserts
    mem_to_reg
48  wire data_write_en = alu_src & ~reg_write & ~mem_to_reg & ~jump
    & ~branch;
49
50  assign pc_out = pc;
51
52  Hardware_Watchdog watchdog_inst (
53      .clk (clk),
54      .rst_in (rst),
55      .PC (pc),
56      .Mode_Bit (mode_bit),
57      .sys_rst_out (watchdog_reset)
58  );
59
60  Control_Unit cu_inst (
61      .opcode (instr[31:26]),
62      .funct (instr[5:0]),
63      .mode_bit (mode_bit),

```

```

64     .zero (zero),
65     .pc (pc),
66     .mem_to_reg (mem_to_reg),
67     .alu_src (alu_src),
68     .reg_dst (reg_dst),
69     .reg_write (reg_write),
70     .jump (jump),
71     .branch (branch),
72     .branch_ne (branch_ne),
73     .alu_control (alu_control),
74     .drop_priv_en (drop_priv_en),
75     .illegal_instr (illegal_instr),
76     .secure_mode_fault (secure_mode_fault),
77     .exception_pc (exception_pc),
78     .exception_taken (cu_exception_taken),
79     .is_lui(is_lui)
80 );
81
82 MIPS_Datapath dp_inst (
83     .clk (clk),
84     .rst (system_reset),
85     .mem_to_reg (mem_to_reg),
86     .alu_src (alu_src),
87     .reg_dst (reg_dst),
88     .reg_write (reg_write),
89     .jump (jump),
90     .branch (branch),
91     .branch_ne (branch_ne),
92     .alu_control (alu_control),
93     .drop_priv_en (drop_priv_en),
94     .illegal_instr (illegal_instr),
95     .secure_mode_fault (secure_mode_fault),

```

```

96     .instr (instr),
97     .pc_out (pc),
98     .read_data (read_data),
99     .alu_result (alu_result),
100    .write_data (write_data),
101    .mode_bit (mode_bit),
102    .exception_taken (dp_exception_taken),
103    .zero (zero),
104    .is_lui(is_lui)
105 );
106
107 assign exception_taken = dp_exception_taken;
108
109 Memory_Router mem_inst (
110     .clk (clk),
111     .rst (system_reset),
112     .mode_bit (mode_bit),
113     .secure_mode_fault (secure_mode_fault),
114     .exception_taken (dp_exception_taken),
115
116     .instr_read_en (instr_read_en),
117     .instr_addr (pc),
118     .instr_allow (instr_allow),
119     .instr_read_data (instr),
120     .instr_violation (instr_violation),
121
122     .data_read_en (data_read_en),
123     .data_read_addr (alu_result),
124     .data_read_allow (data_read_allow),
125     .data_read_data (read_data),
126
127     .data_write_en (data_write_en),

```

```

128     .data_write_addr (alu_result),
129     .data_write_data (write_data),
130     .data_write_allow (data_write_allow),
131
132     .violation_detected (mem_violation_detected)
133 );
134
135 endmodule

```

۴-۱۱ بررسی کد ارائه شده

ساختار کد ماژول سطح بالا را می توان از چند جنبه کلیدی بررسی کرد:

- **مدیریت سیگنال های حافظه (Memory Signals):** از آنجا که طراحی پردازنده به صورت تک سیکلی (Single-Cycle) است، سیگنال واکنشی دستور (instr_read_en) در هر کلاک همواره یک (1'b1) است. سیگنال های فعال ساز خواندن و نوشتن حافظه داده (data_read_en و data_write_en) به کمک سیگنال های کنترلی مسیر داده استخراج می شوند. برای مثال، نوشتن در حافظه زمانی انجام می شود که دستور فعلی از نوع پرش، انشعاب و یا نوشتن در ثبات نباشد.

- **ادغام شبکه های حفاظتی (Protection Mechanisms):** ریست سیستم (system_reset) ترکیبی منطقی (عملگر OR) از ریست کاربر و ریست سخت افزاری (Hardware Watchdog) است. بدین ترتیب در صورت تشخیص نقض امنیتی، تمامی زیرسیستم ها به طور همزمان ریست می شوند.

- **نمونه سازی ماژول ها (Instantiation):** چهار ماژول اصلی به نام های watchdog_inst، cu_inst، dp_inst و mem_inst نمونه سازی شده اند. در این نمونه سازی ها، خروجی هایی مانند alu_result (آدرس دسترسی به داده) و pc (آدرس واکنشی دستور) از Datapath مستقیماً به ورودی های Memory Router متصل شده اند تا چرخه اجرای دستورات کامل شود.

- **سیگنال های خروجی (Output Assignments):** خروجی هایی نظیر pc_out و exception_taken

با استفاده از دستور assign به وضعیت‌های داخلی Datapath متصل شده‌اند تا وضعیت درونی پردازنده برای محیط تست (Testbench) یا سیستم‌های جانبی قابل مشاهده باشد.

۱۲ یکپارچه‌سازی و تست‌بنچ نهایی (Top-Level Testbench)

در این بخش، ماژول پردازنده به همراه تمامی ادوات جانبی مونتاژ شده و برنامه Bootloader اجرا می‌شود.

Listing 7: MIPS_Processor_Top_tb.v

```
1 `timescale 1ns / 1ps
2
3 module tb_MIPS_Top();
4     reg clk, rst;
5     wire [31:0] pc;
6     wire mode_bit;
7     wire exception_taken;
8
9     MIPS_Processor_Top uut (
10         .clk(clk),
11         .rst(rst),
12         .pc_out(pc),
13         .mode_bit(mode_bit),
14         .exception_taken(exception_taken)
15     );
16
17     initial begin
18         clk = 1'b0;
19         rst = 1'b0;
20     end
21     always #5 clk = ~clk;
22
23     integer errors;
24
25     reg [31:0] expected_hash_valid;
26     reg [31:0] expected_hash_tampered;
```

27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52

```
reg saw_drop_priv;
reg entered_user_ram;

initial begin
    errors = 0;
    expected_hash_valid      = 32'h09085427; // 20080005 ^
        20090003 ^ 01095020 ^ 08000401
    expected_hash_tampered = 32'h09085424; // 20080006 ^
        20090003 ^ 01095020 ^ 08000401
    saw_drop_priv = 1'b0;
    entered_user_ram = 1'b0;
end

always @(posedge clk) begin
    if (uut.cu_inst.drop_priv_en)
        saw_drop_priv <= 1'b1;
end

always @(posedge clk) begin
    if (uut.mem_inst.instr_in_user_ram && uut.mem_inst.
        instr_read_en)
        entered_user_ram <= 1'b1;
end

initial begin
    $display("
        =====
    ");
    $display("Starting Secure Boot Testbench");
    $display("
        =====
```

```

    ");
53
54 $display("\n--- Scenario 1: Valid Program ---");
55 rst = 1;
56 #15 rst = 0;
57
58 saw_drop_priv    = 1'b0;
59 entered_user_ram = 1'b0;
60
61 $readmemh("C:/Users/Asus/Downloads/
    CA_Project_kharab_watchdog/CA_Project/user_prog.hex",
62          uut.mem_inst.user_ram);
63
64 #500;
65
66 if (uut.mem_inst.hash_unit.hash_acc === expected_hash_valid)
67     $display("PASS: Computed hash matches reference (%h)",
68             uut.mem_inst.hash_unit.hash_acc);
69 else begin
70     $error("FAIL: Hash mismatch. Got %h, expected %h",
71           uut.mem_inst.hash_unit.hash_acc,
72           expected_hash_valid);
73     errors = errors + 1;
74 end
75
76 if (saw_drop_priv)
77     $display("PASS: DROP_PRIV instruction was executed");
78 else begin
79     $error("FAIL: DROP_PRIV was never asserted");
80     errors = errors + 1;
81 end

```

```

82     if (mode_bit === 1'b0)
83         $display("PASS: Mode dropped to User (mode_bit=0)");
84     else begin
85         $error("FAIL: Mode did NOT drop to User!");
86         errors = errors + 1;
87     end
88
89     if (pc >= 32'h00001000 && pc < 32'h00002000)
90         $display("PASS: PC jumped to User RAM (0x%h)", pc);
91     else begin
92         $error("FAIL: PC not in User RAM (PC=0x%h)", pc);
93         errors = errors + 1;
94     end
95
96     if (entered_user_ram)
97         $display("PASS: Instruction fetch entered User RAM (user
98             program ran)");
99     else begin
100         $error("FAIL: Instruction fetch never entered User RAM")
101             ;
102         errors = errors + 1;
103     end
104
105     if (uut.dp_inst.rf[8] === 32'd5)
106         $display("PASS: $t0 correctly loaded (5)");
107     else begin
108         $error("FAIL: $t0 = %0d, expected 5", uut.dp_inst.rf[8])
109             ;
110         errors = errors + 1;
111     end
112
113     if (uut.dp_inst.rf[9] === 32'd3)

```

```

111         $display("PASS: $t1 correctly loaded (3)");
112     else begin
113         $error("FAIL: $t1 = %0d, expected 3", uut.dp_inst.rf[9])
114         ;
115         errors = errors + 1;
116     end
117
118     if (uut.dp_inst.rf[10] === 32'd8)
119         $display("PASS: User program executed correctly ($t2 =
120             %0d)",
121             uut.dp_inst.rf[10]);
122     else begin
123         $error("FAIL: User program did not compute expected
124             result ($t2 = %0d, expected 8)",
125             uut.dp_inst.rf[10]);
126         errors = errors + 1;
127     end
128
129     $display("\n--- Scenario 2: Tampered Program ---");
130     rst = 1;
131     #15 rst = 0;
132
133     saw_drop_priv      = 1'b0;
134     entered_user_ram    = 1'b0;
135
136     $readmemh("C:/Users/Asus/Downloads/
137         CA_Project_kharab_watchdog/CA_Project/bad_prog.hex",
138         uut.mem_inst.user_ram);
139
140     #500;
141
142     if (uut.mem_inst.hash_unit.hash_acc !== expected_hash_valid)

```

```

139         $display("PASS: Tampered hash correctly differs from
140             reference (%h vs %h)",
141                 uut.mem_inst.hash_unit.hash_acc,
142                 expected_hash_valid);
143
144     else begin
145         $error("FAIL: Tampered hash unexpectedly matches
146             reference!");
147         errors = errors + 1;
148     end
149
150     if (uut.mem_inst.hash_unit.hash_acc ===
151         expected_hash_tampered)
152         $display("PASS: Tampered hash matches expected corrupted
153             value (%h)",
154                 uut.mem_inst.hash_unit.hash_acc);
155     else begin
156         $error("FAIL: Tampered hash = %h, expected %h",
157             uut.mem_inst.hash_unit.hash_acc,
158             expected_hash_tampered);
159         errors = errors + 1;
160     end
161
162     if (mode_bit === 1'b1)
163         $display("PASS: Mode STAYED in Secure (mode_bit=1)");
164     else begin
165         $error("FAIL: Mode dropped to User despite bad hash!");
166         errors = errors + 1;
167     end
168
169     if (pc < 32'h00001000)
170         $display("PASS: PC stayed in Boot ROM (0x%h)", pc);
171     else begin

```

```

165         $error("FAIL: PC escaped to User RAM (PC=0x%h)", pc);
166         errors = errors + 1;
167     end
168
169     if (!saw_drop_priv)
170         $display("PASS: DROP_PRIV correctly never executed on
171                 tampered boot");
172     else begin
173         $error("FAIL: DROP_PRIV executed despite bad hash!");
174         errors = errors + 1;
175     end
176
177     if (!entered_user_ram)
178         $display("PASS: Instruction fetch never entered User RAM
179                 (user program did not run)");
180     else begin
181         $error("FAIL: Instruction fetch entered User RAM despite
182                 bad hash!");
183         errors = errors + 1;
184     end
185
186     $display("\n
187             =====
188             ");
189     if (errors == 0)
190         $display("ALL CHECKS PASSED");
191     else
192         $display("%0d CHECK(S) FAILED", errors);
193     $display("Testbench Finished");
194     $display("
195             =====
196             ");

```

```

190         #50;
191         $finish;
192     end
193
194     always @(posedge clk) begin
195         $display("Time=%5t, PC=%h, Mode=%b, Exc=%b, hash_acc=%h",
196                 $time, pc, mode_bit, exception_taken,
197                 uut.mem_inst.hash_unit.hash_acc);
198     end
199
200 endmodule

```

۱-۱۲ بررسی سناریوهای بالا

این تست‌بنچ عملکرد کلی سیستم Secure Boot را در دو سناریوی اصلی بررسی می‌کند:

- **سناریوی اول (اجرای برنامه معتبر):** در این حالت، برنامه سالم از فایل user_prog.hex در حافظه بارگذاری می‌شود. پردازنده هش را محاسبه کرده و تست‌بنچ درستی آن را با مقدار مرجع (expected_hash_valid) مقایسه می‌کند. پس از تأیید هش، دستور DROP_PRIV با موفقیت اجرا شده، بیت حالت (mode_bit) به صفر (User Mode) تغییر می‌یابد و شمارنده برنامه (PC) به آدرس 0x1000 در حافظه کاربر پرش می‌کند. در نهایت، مقادیر ثبات‌ها (مانند \$t0 تا \$t2) بررسی می‌شوند تا از اجرای صحیح برنامه کاربر اطمینان حاصل شود.

- **سناریوی دوم (اجرای برنامه دستکاری‌شده):** در این حالت، یک برنامه نامعتبر (bad_prog.hex) بارگذاری می‌گردد. تست‌بنچ تأیید می‌کند که هش محاسبه‌شده با هش معتبر تفاوت دارد. در نتیجه، سیستم اجازه اجرای دستور DROP_PRIV را نداده، در حالت Secure باقی می‌ماند و PC هرگز به حافظه کاربر (User RAM) وارد نمی‌شود. این سناریو به خوبی عملکرد مکانیزم دفاعی پردازنده در برابر کدهای مخرب را به اثبات می‌رساند.

۱۳ تست‌بنچ Hardware WatchDog Test Bench

در این بخش، ماژول نگهبان سخت‌افزاری (Hardware Watchdog) به صورت ایزوله مورد آزمایش قرار می‌گیرد تا اطمینان حاصل شود که تحت شرایط مختلف، تخلفات دسترسی به حافظه را به درستی تشخیص داده و سیستم را ریست می‌کند.

Listing 8: tb_Hardware_Watchdog.v

```
1 `timescale 1ns / 1ps
2
3 module tb_Hardware_Watchdog;
4
5     reg clk;
6     reg rst;
7     reg [31:0] pc;
8     reg mode_bit;
9     wire watchdog_reset;
10
11     integer errors = 0;
12
13     Hardware_Watchdog uut (
14         .clk(clk),
15         .rst_in(rst),
16         .PC(pc),
17         .Mode_Bit(mode_bit),
18         .sys_rst_out(watchdog_reset)
19     );
20
21     always #5 clk = ~clk;
22
23     initial begin
24         clk = 0;
25         rst = 1;
```

```

26     pc = 32'h0000_0000;
27     mode_bit = 0;
28
29     #15 rst = 0;
30
31     //
32     -----
33
34     // Test 1: User Mode, PC in RAM -> must NOT reset
35     //
36     -----
37
38     $display("[%0t] Test 1: Normal execution in User Mode",
39             $time);
40     mode_bit = 0;
41     pc = 32'h0000_0004; #10;
42     pc = 32'h0000_0008; #10;
43     pc = 32'h0000_1000; #10;
44
45     if (!watchdog_reset)
46         $display("[%0t] PASS: No reset in User Mode despite PC
47             in RAM.", $time);
48     else begin
49         $display("[%0t] FAIL: Watchdog reset asserted
50             incorrectly in User Mode.", $time);
51         errors = errors + 1;
52     end
53
54     //
55     -----
56
57     // Test 2: Secure Mode, PC in ROM -> must NOT reset

```

```

49 //
    -----

50 $display("[%0t] Test 2: Entering Secure Mode (Valid PC)",
    $time);
51 mode_bit = 1;
52 pc = 32'h0000_0000; #10;
53 pc = 32'h0000_0004; #10;
54 pc = 32'h0000_0008; #10;
55
56 if (!watchdog_reset)
57     $display("[%0t] PASS: No reset in Secure Mode with PC in
        ROM.", $time);
58 else begin
59     $display("[%0t] FAIL: Watchdog reset asserted
        incorrectly with valid Secure PC.", $time);
60     errors = errors + 1;
61 end
62
63 //
    -----

64 // Test 3: Secure Mode, PC illegally in RAM -> MUST reset
65 //
    -----

66 $display("[%0t] Test 3: Security Violation! Secure Mode PC
    in RAM", $time);
67 mode_bit = 1;
68 pc = 32'h0000_1004;
69
70 #20;

```

```

71     if (watchdog_reset)
72         $display("[%0t] PASS: Watchdog detected violation and
           asserted reset.", $time);
73     else begin
74         $display("[%0t] FAIL: Watchdog failed to assert reset on
           violation.", $time);
75         errors = errors + 1;
76     end
77
78     //
       -----
79     // Test 4: External reset applied -> watchdog must self-
       clear
80     //
       -----
81
82     $display("[%0t] Test 4: System Reset applied externally",
           $time);
83     rst = 1;
84     pc = 32'h0000_0000;
85     mode_bit = 0;
86     #10 rst = 0;
87
88     #10; // let combinational logic settle post-reset
89     if (!watchdog_reset)
90         $display("[%0t] PASS: Watchdog cleared cleanly after
           external reset.", $time);
91     else begin
92         $display("[%0t] FAIL: Watchdog reset stuck high after
           external reset.", $time);
93         errors = errors + 1;

```

```

93     end
94
95     //
    -----
96     // Test 5 (new): confirm it stays clear afterward under safe
        conditions
97     //
    -----

98     pc = 32'h0000_0004; #10;
99     pc = 32'h0000_0008; #10;
100    if (!watchdog_reset)
101        $display("[%0t] PASS: Watchdog remains clear during
            normal post-reset execution.", $time);
102    else begin
103        $display("[%0t] FAIL: Watchdog unexpectedly asserted
            after clean reset.", $time);
104        errors = errors + 1;
105    end
106
107    #20;
108    if (errors == 0)
109        $display("=== ALL WATCHDOG TESTS PASSED ===");
110    else
111        $display("=== %0d WATCHDOG TEST(S) FAILED ===", errors);
112
113    $stop;
114 end
115
116 initial begin
117     $monitor("Time=%0t | rst=%b | mode_bit=%b | PC=%h |

```

```

118         watchdog_reset=%b",
119         $time, rst, mode_bit, pc, watchdog_reset);
120
121     end
endmodule

```

۱-۱۳ بررسی سناریوهای بالا

این تست‌بنچ ۵ سناریوی مختلف را برای بررسی صحت عملکرد نگهبان سخت‌افزاری شبیه‌سازی می‌کند:

- **تست ۱ (اجرای عادی در حالت کاربر):** پردازنده در حالت User ($mode_bit=0$) قرار داشته و PC وارد محدوده RAM می‌شود. هیچ ریستی رخ نمی‌دهد که نشان از عملکرد صحیح و مجاز سیستم دارد.
- **تست ۲ (اجرای عادی در حالت امن):** پردازنده در حالت Secure ($mode_bit=1$) بوده و PC در محدوده امن Boot ROM (آدرس‌های پایین) است. در اینجا نیز ریستی اعمال نمی‌گردد.
- **تست ۳ (نقض امنیتی و تشخیص تخلف):** پردازنده در حالت Secure قرار دارد اما PC به صورت غیرمجاز به محدوده User RAM ($0x1004$) وارد می‌شود. نگهبان سخت‌افزاری فوراً این تخلف را تشخیص داده و سیگنال ریست (`watchdog_reset`) را فعال می‌کند تا سیستم متوقف شود.
- **تست ۴ (بازگشت به حالت اولیه با ریست خارجی):** پس از قفل شدن نگهبان در حالت خطا، یک سیگنال ریست خارجی به سیستم اعمال می‌شود. تست‌بنچ بررسی می‌کند که آیا فلیپ‌فلاپ نگهدارنده خطا (Latch) به درستی پاک شده و سیگنال ریست غیرفعال می‌شود یا خیر.
- **تست ۵ (پایداری پس از ریست):** اطمینان حاصل می‌شود که پس از رفع خطای قبلی و اعمال ریست خارجی، سیستم تحت شرایط نرمال و امن مجدداً بدون مشکل کار کرده و ریست ناخواسته‌ای تولید نمی‌کند.

۱۴ خروجی ها و نتایج شبیه سازی

در این بخش تصاویر موج ها (Waveforms) و خروجی های شبیه سازی قرار خواهد گرفت که نشان دهنده عملکرد صحیح سیستم در سناریوهای مختلف است.

۱-۱۴ خروجی transcript

```
# Time=455000, PC=00001004, Mode=0, Exc=0, hash_acc=09085427
# Time=465000, PC=00001008, Mode=0, Exc=0, hash_acc=09085427
# Time=475000, PC=0000100c, Mode=0, Exc=0, hash_acc=09085427
# Time=485000, PC=00001004, Mode=0, Exc=0, hash_acc=09085427
# Time=495000, PC=00001008, Mode=0, Exc=0, hash_acc=09085427
# Time=505000, PC=0000100c, Mode=0, Exc=0, hash_acc=09085427
# PASS: Computed hash matches reference (09085427)
# PASS: DROP_PRIV instruction was executed
# PASS: Mode dropped to User (mode_bit=0)
# PASS: PC jumped to User RAM (0x00001004)
# PASS: Instruction fetch entered User RAM (user program ran)
# PASS: $t0 correctly loaded (5)
# PASS: $t1 correctly loaded (3)
# PASS: User program executed correctly ($t2 = 8)
#
```

شکل ۱: بخش اول خروجی transcript تست بنچ ماژول سطح بالا

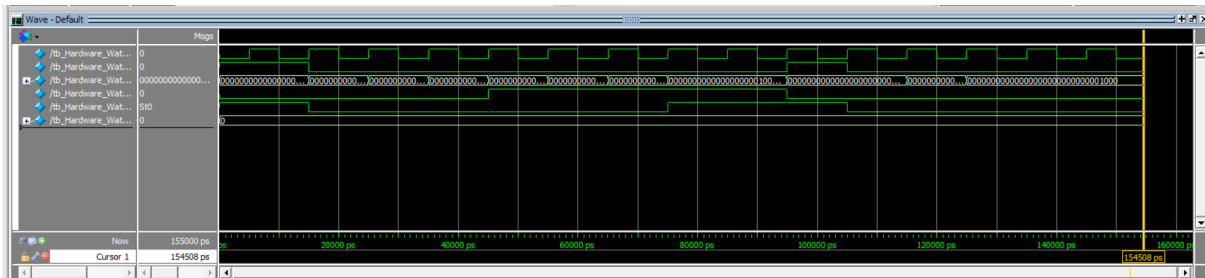
```
=====
# PASS: Tampered hash correctly differs from reference (09085424 vs 09085427)
# PASS: Tampered hash matches expected corrupted value (09085424)
# PASS: Mode STAYED in Secure (mode_bit=1)
# PASS: PC stayed in Boot ROM (0x00000054)
# PASS: DROP_PRIV correctly never executed on tampered boot
# PASS: Instruction fetch never entered User RAM (user program did not run)
#
# =====
# ALL CHECKS PASSED
# Testbench Finished
# =====
# Time=1035000, PC=00000054, Mode=1, Exc=1, hash_acc=09085424
# Time=1045000, PC=00000054, Mode=1, Exc=1, hash_acc=09085424
# Time=1055000, PC=00000054, Mode=1, Exc=1, hash_acc=09085424
# Time=1065000, PC=00000054, Mode=1, Exc=1, hash_acc=09085424
# Time=1075000, PC=00000054, Mode=1, Exc=1, hash_acc=09085424
# ** Note: $finish : C:/Users/Asus/Downloads/GA_Project_kharab_watchdog/GA_Project/MIPS_Processor_Top_tb.v(267)
# Time: 1080 ns Iteration: 0 Instance: /tb_MIPS_Top
#
```

شکل ۲: بخش اول خروجی transcript تست بنچ ماژول سطح بالا

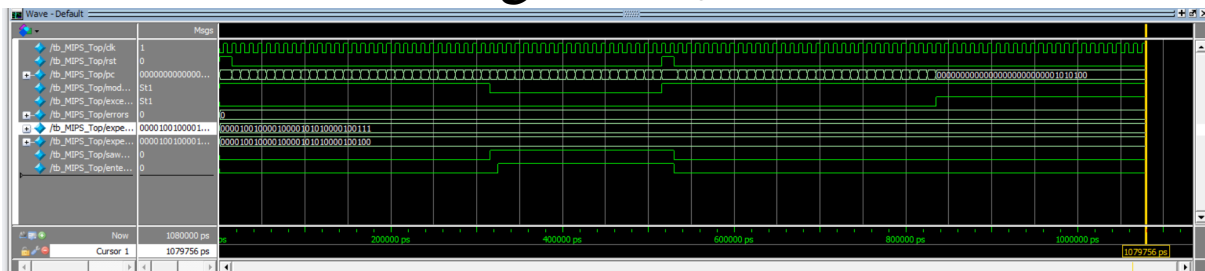
```
# Time=0 | rst=1 | mode_bit=0 | PC=00000000 | watchdog_reset=1
# [15000] Test 1: Normal execution in User Mode
# Time=15000 | rst=0 | mode_bit=0 | PC=00000004 | watchdog_reset=0
# Time=25000 | rst=0 | mode_bit=0 | PC=00000008 | watchdog_reset=0
# Time=35000 | rst=0 | mode_bit=0 | PC=00001000 | watchdog_reset=0
# [45000] PASS: No reset in User Mode despite PC in RAM.
# [45000] Test 2: Entering Secure Mode (Valid PC)
# Time=45000 | rst=0 | mode_bit=1 | PC=00000000 | watchdog_reset=0
# Time=55000 | rst=0 | mode_bit=1 | PC=00000004 | watchdog_reset=0
# Time=65000 | rst=0 | mode_bit=1 | PC=00000008 | watchdog_reset=0
# [75000] PASS: No reset in Secure Mode with PC in ROM.
# [75000] Test 3: Security Violation! Secure Mode PC in RAM
# Time=75000 | rst=0 | mode_bit=1 | PC=00001004 | watchdog_reset=1
# [95000] PASS: Watchdog detected violation and asserted reset.
# [95000] Test 4: System Reset applied externally
# Time=95000 | rst=1 | mode_bit=0 | PC=00000000 | watchdog_reset=1
# Time=105000 | rst=0 | mode_bit=0 | PC=00000000 | watchdog_reset=0
# [115000] PASS: Watchdog cleared cleanly after external reset.
# Time=115000 | rst=0 | mode_bit=0 | PC=00000004 | watchdog_reset=0
# Time=125000 | rst=0 | mode_bit=0 | PC=00000008 | watchdog_reset=0
# [135000] PASS: Watchdog remains clear during normal post-reset execution.
# === ALL WATCHDOG TESTS PASSED ===
```

شکل ۳: خروجی transcript بخش WatchDog

۱۴-۲ خروجی موج



شکل ۴: خروجی موج اول



شکل ۵: خروجی موج دوم

۱۵ چالش‌ها و مشکلات پروژه

۱-۱۵ چالش‌های اشکال‌زدایی تست‌بنچ راه‌اندازی امن

در مرحله نخست پیاده‌سازی، با وجود اینکه معماری کلی پردازنده (واحد کنترل، مسیر داده، Mem-ory Router و شتاب‌دهنده هش) به‌درستی طراحی و تست واحد شده بود، سناریوی اول تست‌بنچ (بوت موفق) به‌طور مداوم شکست می‌خورد. بررسی دقیق موج‌های شبیه‌سازی نشان داد که ریشه مشکل نه در منطق پردازنده، بلکه در نحوه تولید سیگنال کلاک در تست‌بنچ بود: متغیر `clk` بدون مقداردهی اولیه تعریف شده بود و از آنجا که در Verilog مقدار پیش‌فرض رجیسترها `x` (نامشخص) است، عبارت `clk = clk` هرگز باعث تغییر واقعی مقدار سیگنال نمی‌شد و در نتیجه هیچ‌گاه لبه بالارونده معتبری تولید نمی‌گشت. این موضوع باعث می‌شد کل پردازنده، به‌جز آنچه در همان لحظه اعمال ریست بیرونی رخ می‌داد، عملاً منجمد بماند. رفع این اشکال با مقداردهی اولیه صریح `clk = 1'b0` امکان‌پذیر شد.

پس از رفع مشکل کلاک، اشکال دوم آشکار شد: برنامه راه‌انداز (Bootloader) ذخیره‌شده در Boot ROM از دستورات `addi` استفاده می‌کرد، اما کدهای ماشین تولیدشده در عمل از کد عملیات `0x09` (معادل `ADDIU` در معماری استاندارد MIPS) استفاده می‌کردند، درحالی‌که واحد کنترل ما تنها کد عملیات `0x08` را به‌عنوان `ADDI` پیاده‌سازی کرده بود. این ناهم‌خوانی باعث می‌شد همان اولین دستور جمع در برنامه راه‌انداز به‌عنوان دستور نامعتبر تشخیص داده شود و پردازنده وارد وضعیت استثنا و توقف دائمی PC گردد. با اصلاح کدگذاری این دستورات در Boot ROM، این مشکل نیز برطرف شد.

چالش سوم به کارایی حلقه محاسبه هش بازمی‌گشت: محدوده حلقه در برنامه راه‌انداز به‌گونه‌ای تنظیم شده بود که ۲۵۶ بار تکرار می‌شد، درحالی‌که برنامه کاربر تنها ۴ کلمه داشت. از آنجا که XOR کردن مقادیر صفر تأثیری در نتیجه نهایی هش ندارد، این تکرار اضافی از نظر منطقی بی‌خطر بود، اما از نظر زمانی هزاران سیکل کلاک غیرضروری به شبیه‌سازی تحمیل می‌کرد و باعث می‌شد بازه زمانی در نظر گرفته‌شده در تست‌بنچ برای رسیدن به دستور `DROP_PRIV` کافی نباشد. با اصلاح محدوده حلقه متناسب با طول واقعی برنامه کاربر، زمان اجرا به‌طور چشمگیری کاهش یافت.

آخرین و ظریف‌ترین مشکل، مربوط به لحظه گذار از حالت امن به حالت کاربر بود. در طراحی اولیه، اجرای دستور `DROP_PRIV` تنها بیت `Mode` را تغییر می‌داد و پرش به آدرس برنامه کاربر برعهده یک دستور جامپ مجزا در ادامه برنامه راه‌انداز بود. اما دقیقاً در همان سیکلی که بیت `Mode`

به کاربر تغییر می‌کرد، پردازنده هنوز یک واکنشی دستور از داخل Boot ROM را در پیش داشت که در حالت کاربر مجاز نبود؛ این امر باعث بروز استثنای دسترسی غیرمجاز درست در رمز گذار حالت می‌شد. راه‌حل نهایی این بود که خود دستور DROP_PRIV به صورت اتمیک، هم بیت Mode را تغییر دهد و هم مستقیماً PC را به آدرس ورودی برنامه کاربر (0x1000) هدایت کند، بدون نیاز به واکنشی دستور میانی از حافظه راه‌انداز.

۱۵-۲ چالش‌های افزودن واحد Hardware Execution Watchdog

در گام افزودن واحد نگهبان سخت‌افزاری (Watchdog)، نخستین مانع از نوع ساختاری و نه منطقی بود: نام پورت‌های استفاده‌شده در نمونه‌سازی ماژول در سطح بالا (pc, rst, mode_bit, watchdog_reset) با نام واقعی پورت‌های تعریف‌شده در ماژول Hardware_Watchdog (rst_in, PC, Mode_Bit, sys_rst_out) مطابقت نداشت. از آنجا که Verilog نسبت به بزرگی و کوچکی حروف حساس است، این عدم تطابق باعث بروز خطای Elaboration در شبیه‌ساز شد و پیش از هرگونه بررسی رفتاری، باید نام‌گذاری اتصالات اصلاح می‌شد.

پس از رفع خطای اتصال، چالش دقیق‌تری در طراحی منطق داخلی Watchdog آشکار شد. در نسخه نخست، سیگنال خروجی ریست به صورت یک رجیستر همزمان (synchronous) پیاده‌سازی شده بود که تنها روی لبه بالارونده کلاک به‌روزرسانی می‌شد. این طراحی به دو دلیل مشکل‌ساز بود: نخست آنکه با یک سیکل تأخیر نسبت به لحظه وقوع واقعی تخلف امنیتی فعال می‌شد که با فلسفه «پایش کاملاً سخت‌افزاری و بدون وابستگی به نرم‌افزار» مطرح‌شده در صورت پروژه هم‌خوانی کامل نداشت؛ دوم آنکه رجیستر ریست ورودی خود Watchdog (rst_in) تنها به ریست بیرونی متصل بود، اما در برخی حالت‌ها لازم بود این واحد بتواند وضعیت تخلف را به صورت پایدار (Latch) نگه دارد تا با از بین رفتن لحظه‌ای شرط نقض (مثلاً پس از اعمال ریست به پردازنده)، سیگنال ریست زودتر از موعد قطع نشود. راه‌حل نهایی ترکیب دو مکانیزم بود: یک مسیر ترکیبی (Combinational) مستقیم که بدون هیچ تأخیری هم‌زمان با وقوع تخلف سیگنال ریست را فعال می‌کند، همراه با یک بیت Latch ثبت‌شده که تضمین می‌کند حتی اگر شرط نقض تنها برای یک سیکل برقرار باشد، سیستم به‌طور کامل و پایدار بازنشانی شود.

مسئله دیگری که در طول این مرحله به آن توجه ویژه شد، اطمینان از عدم بروز هشدار کاذب (False Positive) توسط Watchdog در روند بوت عادی و موفق بود. از آنجا که در طراحی نهایی، اجرای DROP_PRIV به صورت اتمیک هم‌زمان بیت Mode را به کاربر تغییر می‌دهد و هم PC را به ابتدای RAM کاربر منتقل می‌کند، هیچ‌گاه شرط «PC در محدوده RAM و هم‌زمان Mode

برابر با Secure» به طور هم زمان در طول اجرای صحیح برنامه برقرار نمی شود؛ این ویژگی به صورت مستقیم نتیجه رفع مشکل قبلی (گذار اتمیک حالت) بود و نشان می دهد که دو بخش از پروژه، با وجود پیاده سازی مستقل، از نظر رفتاری به یکدیگر وابسته اند. برای اطمینان از عملکرد صحیح Watchdog در برابر تخلف واقعی، سناریوی آزمایشی جداگانه ای طراحی شد که با تزریق عمدی یک مقدار نامعتبر به PC در حالت امن (شبیه سازی یک حمله یا خطای سخت افزاری)، فعال سازی صحیح سیگنال ریست کل سیستم و بازیابی پردازنده به وضعیت امن اولیه را تأیید کرد.

۱۵-۳ چالش های مرتبط با مسیر داده (DataPath)

چالش اصلی در طراحی مسیر داده، مدیریت صحیح سیگنال استثنا (exception_taken) بود. در نسخه اولیه، این سیگنال به صورت یک رجیستر جداگانه تعریف شده بود که با یک سیکل تأخیر نسبت به تشخیص خطا توسط واحد کنترل، به روزرسانی می شد. این تأخیر باعث می شد PC یک قدم اضافی به جلو حرکت کند و دستور بعدی نیز واکنشی شود. راه حل نهایی، حذف رجیستر و استفاده از یک سیگنال ترکیبی (combinational) مستقیم از خروجی واحد کنترل بود تا PC بلافاصله متوقف شود.

چالش دیگر مربوط به دستورات پرش شرطی (BEQ و BNE) بود. در طراحی اولیه، تنها سیگنال zero برای تصمیم گیری استفاده می شد که برای BNE کافی نبود. با اضافه کردن سیگنال branch_ne و پیاده سازی منطق zero : zero ? branch_ne = branch_condition، هر دو نوع پرش به درستی پشتیبانی شدند.

همچنین در فایل ثبات ها (Register File)، مشکل مربوط به ثبات صفر وجود داشت. با اضافه کردن شرط write_reg != 5'b0 در زمان نوشتن، از تغییر مقدار این ثبات جلوگیری شد.